

Phase 4: Better Probing — Quadratic Probing and Double Hashing

1. Current Progress Analysis

Before Phase 4, you already learned the main foundation of hashing.

In **Phase 1**, you learned that hashing uses a formula to turn a key into an index.

Key → Hash Function → Index

Example:

$$H(25) = 25 \bmod 10 = 5$$

This means key `25` wants to be stored at index `5`.

In **Phase 2**, you learned **open hashing with chaining**.

In chaining, if many keys go to the same index, we store them in a linked list.

Example:

Index 5: 5 → 25 → 75 → NULL

In **Phase 3**, you learned **closed hashing with linear probing**.

In linear probing, all keys stay inside the original array. If a key finds its index full, it checks the next slot, then the next slot, and so on.

Example:

```
Try index 5
If full, try index 6
If full, try index 7
If full, try index 8
...
```

Linear probing is simple, but it has a big weakness:

Primary clustering

Primary clustering means many keys form a long continuous block in the table.

Example:

Index:	3	4	5	6	7	8	9
Value:	23	43	45	26	25	74	19

This long block makes searching and insertion slower.

So Phase 4 teaches better probing methods:

1. Quadratic probing
2. Double hashing

Both methods try to reduce clustering by using smarter probe jumps.

2. Main Goal of Phase 4

The main goal of Phase 4 is to improve collision handling inside a closed hash table.

In this phase, you learn:

- What quadratic probing means
- Why quadratic probing is better than simple linear probing
- The quadratic probing formula
- How square jumps work
- A full quadratic probing dry run
- The loading factor rule for quadratic probing
- The average search formulas given in the curriculum
- What double hashing means
- Why double hashing uses two hash functions
- The primary hash function $H_1(x)$
- The secondary hash function $H_2(x)$
- Rules for the secondary hash function
- The double hashing formula
- A full double hashing dry run
- How quadratic probing and double hashing reduce clustering
- Complete C code for both methods
- Common beginner mistakes

The main beginner idea is:

Instead of moving one step after a collision, use smarter jumps.

3. What Problem Does Phase 4 Solve?

Phase 4 solves the problem caused by **linear probing**.

In linear probing, after a collision, we move one slot at a time.

Formula:

$$H'(x) = (H(x) + i) \bmod \text{size}$$

For linear probing:

$$i = 0, 1, 2, 3, 4, \dots$$

So the movement is:

$$+0, +1, +2, +3, +4, \dots$$

This creates long continuous groups of filled slots.

Example:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	30	29	_	23	43	45	26	25	74	19

The block from index **3** to index **9** is a cluster.

This is bad because if a new key hashes anywhere inside that block, it must walk through many full slots before finding an empty one.

Phase 4 improves this by changing the probe sequence.

Instead of:

Check next, next, next, next

Phase 4 uses:

Quadratic probing: jump by squares
Double hashing: jump using a second hash function

4. Closed Hashing Reminder

Both quadratic probing and double hashing are types of **closed hashing**.

Closed hashing means:

All keys must stay inside the original hash table array.

There are no linked lists.

If a collision happens, the algorithm must find another empty slot inside the same array.

Simple comparison:

Method	Collision solution	Extra linked-list memory?
Chaining	Put colliding keys in a linked list	Yes
Linear probing	Move to another slot in the array	No
Quadratic probing	Jump by square values inside the array	No
Double hashing	Use a second hash function to jump inside the array	No

Part A: Quadratic Probing

5. What Is Quadratic Probing?

Quadratic probing is a closed hashing collision-resolution method.

It is similar to linear probing because everything stays inside the array.

But the difference is in the jump pattern.

Linear probing moves like this:

+0, +1, +2, +3, +4, ...

Quadratic probing moves like this:

+0², +1², +2², +3², +4², ...

That gives:

+0, +1, +4, +9, +16, ...

So quadratic probing does not always check the immediate next slot.

It jumps farther as i increases.

Simple meaning:

Quadratic probing = collision handling using square jumps.

6. Why Is It Called Quadratic?

It is called **quadratic** because it uses i^2 .

The word quadratic means something involving a square.

Examples:

$0^2 = 0$
 $1^2 = 1$
 $2^2 = 4$
 $3^2 = 9$
 $4^2 = 16$

So when we probe, we use square numbers to calculate the next index.

7. Quadratic Probing Formula

The formula is:

$$H'(x) = (H(x) + i^2) \bmod \text{size}$$

Where:

Symbol	Meaning
x	The key
$H(x)$	The original hash index
i	Probe number
i^2	Square jump
size	Hash table size
$\bmod \text{size}$	Keeps the index inside the table

The values of i are:

$$i = 0, 1, 2, 3, 4, \dots$$

Important:

i starts at 0.

Why?

Because the first attempt should be the original hash index.

When $i = 0$:

$$\begin{aligned} H'(x) &= (H(x) + 0^2) \bmod \text{size} \\ H'(x) &= H(x) \bmod \text{size} \end{aligned}$$

So the first check is the normal hash position.

8. Quadratic Probing Probe Sequence

Suppose:

Table size = 10
 $H(x) = 3$

Then quadratic probing checks:

Probe number i	Formula	Index checked
0	$(3 + 0^2) \bmod 10$	3
1	$(3 + 1^2) \bmod 10$	4
2	$(3 + 2^2) \bmod 10$	7
3	$(3 + 3^2) \bmod 10$	2
4	$(3 + 4^2) \bmod 10$	9

So the probe path is:

$3 \rightarrow 4 \rightarrow 7 \rightarrow 2 \rightarrow 9 \rightarrow \dots$

Notice that it does not simply go:

$3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

That simple one-by-one movement is linear probing, not quadratic probing.

9. Why Quadratic Probing Reduces Primary Clustering

Primary clustering happens when many filled slots become connected in one long block.

Linear probing creates primary clustering because every collision moves to the next slot.

Example:

Try 5
Then 6
Then 7

Then 8
Then 9

This can create a continuous block.

Quadratic probing reduces this because it jumps by square numbers.

Example:

Try 5
Then 6
Then 9
Then 4
Then 1

The keys are more spread out.

Simple meaning:

Quadratic probing avoids always walking through the same crowded line.

10. Quadratic Probing Real-Life Analogy

Imagine a parking lot.

In linear probing:

If space 5 is full, check 6.
If 6 is full, check 7.
If 7 is full, check 8.

You walk one space at a time.

In quadratic probing:

Try space 5.
If full, jump 1 space.
If full, jump 4 spaces.
If full, jump 9 spaces.

You are not just walking to the next space each time.

You are making bigger jumps.

This can help avoid crowded areas.

11. Quadratic Probing Insertion Steps

To insert a key using quadratic probing:

1. Take the key.
2. Calculate the original hash index using $H(x)$.
3. If that index is empty, insert the key there.
4. If it is full, try the quadratic formula with $i = 1$.
5. If still full, try $i = 2$.
6. Continue using i^2 until an empty slot is found.
7. Use mod size so the index stays inside the table.

Simple rule:

Insert at the first empty slot found by the quadratic probe sequence.

12. Quadratic Probing Dry Run

Use:

Table size = 10
Hash function: $H(x) = x \bmod 10$
Keys: 23, 43, 13, 27

Start with an empty table:

Index: 0 1 2 3 4 5 6 7 8 9
Value: _ _ _ _ _ _ _ _ _ _

12.1 Insert 23

Calculate:

$$23 \bmod 10 = 3$$

Index **3** is empty.

Store **23** at index **3**.

Table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	23	_	_	_	_	_	_

12.2 Insert 43

Calculate:

$$43 \bmod 10 = 3$$

Index **3** already contains **23**.

This is a collision.

Use quadratic probing:

i	=	0	:	(3 + 0 ²)	mod	10	=	3	→	full
i	=	1	:	(3 + 1 ²)	mod	10	=	4	→	empty

Store **43** at index **4**.

Table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	23	43	_	_	_	_	_

12.3 Insert 13

Calculate:

$$13 \bmod 10 = 3$$

Index 3 is full.

Use quadratic probing:

$i = 0: (3 + 0^2) \bmod 10 = 3 \rightarrow \text{full}$
 $i = 1: (3 + 1^2) \bmod 10 = 4 \rightarrow \text{full}$
 $i = 2: (3 + 2^2) \bmod 10 = 7 \rightarrow \text{empty}$

Store 13 at index 7.

Table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	23	43	_	_	13	_	_

Important observation:

13 did not go to index 5 or 6.

It jumped from index 4 to index 7 because:

$$2^2 = 4$$
$$3 + 4 = 7$$

12.4 Insert 27

Calculate:

$$27 \bmod 10 = 7$$

Index 7 already contains 13.

This is a collision.

Use quadratic probing:

$i = 0: (7 + 0^2) \bmod 10 = 7 \rightarrow \text{full}$
 $i = 1: (7 + 1^2) \bmod 10 = 8 \rightarrow \text{empty}$

Store 27 at index 8.

Final table:

Index: 0 1 2 3 4 5 6 7 8 9
Value: _ _ _ 23 43 _ _ 13 27 _

13. Quadratic Probing Dry Run Table

Key	Initial hash	Probe path	Stored at
23	3	Index 3 free	3
43	3	3 full, try 4	4
13	3	3 full, 4 full, try 7	7
27	7	7 full, try 8	8

Main lesson:

Quadratic probing uses square jumps, so it spreads keys more than linear probing.

14. Searching in Quadratic Probing

Searching follows the same probe path as insertion.

Search steps:

1. Take the search key.
2. Calculate the original hash index.
3. Check that index.
4. If the key is found, stop successfully.
5. If the slot has another key, continue using quadratic probing.
6. If an empty slot is found, stop and say the key is not present.

Important:

Search must use the same formula that insertion used.

So if insertion used:

$$H'(x) = (H(x) + i^2) \bmod \text{size}$$

Then search must also use:

$$H'(x) = (H(x) + i^2) \bmod \text{size}$$

15. Quadratic Probing Search Example

Use the final table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	23	43	_	_	13	27	_

Search for 13.

Calculate:

$$13 \bmod 10 = 3$$

Search path:

i = 0: index 3 contains 23, not 13
i = 1: index 4 contains 43, not 13
i = 2: index 7 contains 13, found

Result:

13 found at index 7

Search for 33.

Calculate:

$$33 \bmod 10 = 3$$

Search path:

```
i = 0: index 3 contains 23, not 33  
i = 1: index 4 contains 43, not 33  
i = 2: index 7 contains 13, not 33  
i = 3: index 2 is empty
```

Result:

33 not found

Why stop at empty?

Because if `33` had been inserted, it would have followed the same quadratic path and stopped at that empty slot.

16. Loading Factor in Quadratic Probing

The loading factor is:

$$\lambda = n / \text{size}$$

Where:

Symbol	Meaning
<code>lambda</code>	Loading factor
<code>n</code>	Number of stored keys
<code>size</code>	Hash table size

The curriculum gives this rule for quadratic probing:

$$\lambda \leq 0.5$$

Simple meaning:

Keep the table at most half full.

Example:

Table size = 10
Maximum recommended keys ≈ 5

Why?

Because quadratic probing needs empty slots to work well.

If the table is too full:

- Collisions become more common
- Probe sequences become longer
- Search becomes slower
- Insertion becomes slower
- Some probe sequences may fail to find an empty slot quickly

17. Average Search Formulas for Quadratic Probing

The curriculum gives these formulas:

Operation	Formula
Successful search	$-\log_e(1 - \lambda) - \lambda / 2$
Unsuccessful search	$1 / (1 - \lambda) - \lambda - \ln(1 - \lambda)$

As a beginner, you do not need to derive these formulas.

The simple idea is:

As λ increases, the table gets fuller.
As the table gets fuller, search needs more probes.

So remember:

Lower λ = faster probing
Higher λ = slower probing

18. Advantages of Quadratic Probing

Quadratic probing has these advantages:

- It stores all keys inside the array.
- It does not need linked lists.
- It reduces primary clustering compared with linear probing.
- It spreads keys better than simple one-step movement.
- It is still based on a simple formula.

19. Limitations of Quadratic Probing

Quadratic probing also has limitations:

- It still needs empty space in the table.
- It works best when the loading factor is not high.
- It can still have probing problems depending on the table size.
- Keys with the same starting index still follow the same quadratic pattern.
- It reduces primary clustering but does not completely remove all clustering issues.

Simple meaning:

Quadratic probing is better than linear probing, but it is not perfect.

Part B: Double Hashing

20. What Is Double Hashing?

Double hashing is another closed hashing collision-resolution method.

It uses two hash functions:

$H_1(x)$ = first hash function
 $H_2(x)$ = second hash function

The first hash function gives the starting index.

The second hash function gives the jump size.

Simple meaning:

Double hashing = use one hash function to start and another hash function to jump.

21. Why Double Hashing Is Useful

In linear probing, the jump size is always 1.

In quadratic probing, the jump pattern is based on square numbers.

But in double hashing, the jump size depends on the key itself.

This means different keys can follow different probe paths.

That is powerful because even if many keys first collide at the same index, they may move in different directions after the collision.

Simple idea:

Same starting index does not always mean same full path.

22. Double Hashing Formula

The formula is:

$$H'(x) = (H1(x) + i * H2(x)) \bmod \text{size}$$

Where:

Symbol	Meaning
x	The key
H1(x)	Primary hash function / starting index
H2(x)	Secondary hash function / jump size
i	Probe number

Symbol	Meaning
size	Hash table size

The values of i are:

$i = 0, 1, 2, 3, \dots$

When $i = 0$:

$H'(x) = (H1(x) + 0 * H2(x)) \bmod \text{size}$
 $H'(x) = H1(x)$

So the first attempt is always the original index from $H1(x)$.

23. Hash Functions Used in the Curriculum Example

The curriculum example uses table size 10.

Primary hash function:

$H1(x) = x \bmod 10$

Secondary hash function:

$H2(x) = 7 - (x \bmod 7)$

Why use 7?

Because 7 is a prime number smaller than 10.

A prime number is often used in the second hash function because it helps create better probe movement.

24. Rules for the Secondary Hash Function

The secondary hash function $H2(x)$ is very important.

Rule 1:

$H_2(x)$ must never return 0.

Why?

Because if the jump size is 0, the algorithm keeps checking the same index forever.

Example:

$$H'(x) = (H_1(x) + i * 0) \bmod \text{size}$$
$$H'(x) = H_1(x)$$

No movement happens.

Rule 2:

$H_2(x)$ should help reach many table positions.

A poor second hash function can make double hashing fail or behave badly.

Rule 3:

$H_2(x)$ should depend on the key.

This is what gives different keys different jump sizes.

25. Double Hashing Real-Life Analogy

Imagine a parking lot.

In linear probing:

If your space is full, move one space each time.

In quadratic probing:

If your space is full, jump by square distances.

In double hashing:

Each car has its own jump size.

Example:

Car A jumps by 3 spaces.
Car B jumps by 6 spaces.
Car C jumps by 7 spaces.

So cars do not all follow the same path.

That reduces crowding.

26. Double Hashing Insertion Steps

To insert a key using double hashing:

1. Take the key.
2. Calculate $H1(\text{key})$ to find the starting index.
3. If that index is empty, insert the key there.
4. If it is full, calculate $H2(\text{key})$ to find the jump size.
5. Use the formula:

$$H'(x) = (H1(x) + i * H2(x)) \bmod \text{size}$$

1. Increase i until an empty slot is found.
 2. Insert the key into the first empty slot in that probe path.
-

27. Double Hashing Dry Run

Use:

Table size = 10
 $H1(x) = x \bmod 10$
 $H2(x) = 7 - (x \bmod 7)$
Keys: 5, 25, 15, 35, 95

Start with an empty table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	_	_	_	_	_	_	_

All keys end in **5**, so all of them have the same first hash index:

5 mod 10	=	5
25 mod 10	=	5
15 mod 10	=	5
35 mod 10	=	5
95 mod 10	=	5

So all keys first want index **5**.

This is a good example because it shows how double hashing handles repeated collisions.

27.1 Insert 5

Calculate:

$H_1(5) = 5 \bmod 10 = 5$

Index **5** is empty.

Store **5** at index **5**.

Table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	_	_	5	_	_	_	_

27.2 Insert 25

Calculate the first hash:

$H_1(25) = 25 \bmod 10 = 5$

Index **5** is full.

Now calculate the second hash:

$$\begin{aligned}H_2(25) &= 7 - (25 \bmod 7) \\ 25 \bmod 7 &= 4 \\ H_2(25) &= 7 - 4 = 3\end{aligned}$$

So the jump size is .

Probe:

$$\begin{aligned}i = 0: (5 + 0 * 3) \bmod 10 &= 5 \rightarrow \text{full} \\ i = 1: (5 + 1 * 3) \bmod 10 &= 8 \rightarrow \text{empty}\end{aligned}$$

Store at index .

Table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	_	_	5	_	_	25	_

27.3 Insert 15

Calculate the first hash:

$$H_1(15) = 15 \bmod 10 = 5$$

Index is full.

Calculate the second hash:

$$\begin{aligned}H_2(15) &= 7 - (15 \bmod 7) \\ 15 \bmod 7 &= 1 \\ H_2(15) &= 7 - 1 = 6\end{aligned}$$

So the jump size is .

Probe:

$i = 0: (5 + 0 * 6) \bmod 10 = 5 \rightarrow \text{full}$
 $i = 1: (5 + 1 * 6) \bmod 10 = 11 \bmod 10 = 1 \rightarrow \text{empty}$

Store 15 at index 1.

Table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	15	_	_	_	5	_	_	25	_

27.4 Insert 35

Calculate the first hash:

$H1(35) = 35 \bmod 10 = 5$

Index 5 is full.

Calculate the second hash:

$H2(35) = 7 - (35 \bmod 7)$
 $35 \bmod 7 = 0$
 $H2(35) = 7 - 0 = 7$

So the jump size is 7.

Probe:

$i = 0: (5 + 0 * 7) \bmod 10 = 5 \rightarrow \text{full}$
 $i = 1: (5 + 1 * 7) \bmod 10 = 12 \bmod 10 = 2 \rightarrow \text{empty}$

Store 35 at index 2.

Table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	15	35	_	_	5	_	_	25	_

27.5 Insert 95

Calculate the first hash:

$$H1(95) = 95 \bmod 10 = 5$$

Index **5** is full.

Calculate the second hash:

$$\begin{aligned} H2(95) &= 7 - (95 \bmod 7) \\ 95 \bmod 7 &= 4 \\ H2(95) &= 7 - 4 = 3 \end{aligned}$$

So the jump size is **3**.

Probe:

$$\begin{aligned} i = 0: (5 + 0 * 3) \bmod 10 &= 5 \rightarrow \text{full} \\ i = 1: (5 + 1 * 3) \bmod 10 &= 8 \rightarrow \text{full} \\ i = 2: (5 + 2 * 3) \bmod 10 &= 11 \bmod 10 = 1 \rightarrow \text{full} \\ i = 3: (5 + 3 * 3) \bmod 10 &= 14 \bmod 10 = 4 \rightarrow \text{empty} \end{aligned}$$

Store **95** at index **4**.

Final table:

```
Index: 0 1 2 3 4 5 6 7 8 9
Value: _ 15 35 _ 95 5 _ _ 25 _
```

28. Double Hashing Dry Run Table

Key	H1	H2	Probe path	Stored at
5	5	Not needed	Index 5 free	5
25	5	3	5 full, 8 free	8
15	5	6	5 full, 1 free	1
35	5	7	5 full, 2 free	2

Key	H1	H2	Probe path	Stored at
95	5	3	5 full, 8 full, 1 full, 4 free	4

Main lesson:

Even if keys collide at the same starting index, double hashing can send them to different places.

29. Searching in Double Hashing

Searching follows the same path as insertion.

Search steps:

1. Take the search key.
2. Calculate $H1(key)$.
3. Check that index.
4. If the key is there, search is successful.
5. If another key is there, calculate $H2(key)$.
6. Follow the same jump sequence.
7. If the key is found, return success.
8. If an empty slot is found, return not found.

Important:

The same key must produce the same H1 and H2 values during insertion and search.

30. Double Hashing Search Example

Use the final table:

Index: 0 1 2 3 4 5 6 7 8 9
 Value: _ 15 35 _ 95 5 _ _ 25 _

Search for 95.

Calculate:

```
H1(95) = 95 mod 10 = 5
H2(95) = 7 - (95 mod 7) = 3
```

Search path:

```
i = 0: index 5 contains 5, not 95
i = 1: index 8 contains 25, not 95
i = 2: index 1 contains 15, not 95
i = 3: index 4 contains 95, found
```

Result:

95 found at index 4

Search for 45.

Calculate:

```
H1(45) = 45 mod 10 = 5
H2(45) = 7 - (45 mod 7)
45 mod 7 = 3
H2(45) = 4
```

Search path:

```
i = 0: index 5 contains 5, not 45
i = 1: (5 + 1 * 4) mod 10 = 9 → empty
```

Result:

45 not found

31. Why Double Hashing Reduces Clustering

Double hashing reduces clustering because each key can have a different jump size.

Example:

```
25 has jump size 3
15 has jump size 6
35 has jump size 7
95 has jump size 3
```

So even though all of them first wanted index `5`, they did not all move in the same way.

Simple comparison:

```
Linear probing: everyone moves +1
Quadratic probing: everyone uses square jumps
Double hashing: each key gets its own jump size
```

That is why double hashing usually spreads keys better.

32. Advantages of Double Hashing

Double hashing has these advantages:

- It stores all keys inside the array.
 - It does not need linked lists.
 - It reduces clustering better than linear probing.
 - It can reduce collision patterns better than quadratic probing.
 - Different keys can follow different probe paths.
 - It is useful when many keys collide at the same first index.
-

33. Limitations of Double Hashing

Double hashing also has limitations:

- It needs a good second hash function.
- `H2(x)` must never be zero.
- Poor table size choices can create bad probe sequences.
- It is slightly more complex than linear or quadratic probing.
- If the table is too full, it can still become slow.

Simple meaning:

```
Double hashing is powerful, but only if H2(x) is designed well.
```

Part C: Comparison of Methods

34. Linear Probing vs Quadratic Probing vs Double Hashing

Feature	Linear probing	Quadratic probing	Double hashing
Collision movement	One step at a time	Square jumps	Key-based jump size
Formula	$(H(x) + i) \bmod \text{size}$	$(H(x) + i^2) \bmod \text{size}$	$(H1(x) + i * H2(x)) \bmod \text{size}$
Main advantage	Simple	Reduces primary clustering	Usually spreads keys best
Main weakness	Primary clustering	Probe sequence issues	Needs good $H2(x)$
Uses linked lists?	No	No	No
Closed hashing?	Yes	Yes	Yes

35. Memory Trick

Use this simple memory trick:

Linear probing:
Next, next, next

Quadratic probing:
Jump by squares

Double hashing:
Second hash function decides the jump

Another way:

Linear = walking
Quadratic = jumping
Double hashing = each key has its own jumping rule

36. Which Method Is Better?

It depends on the situation.

Linear probing is easiest, but it creates primary clustering.

Quadratic probing is better because it jumps by squares and reduces primary clustering.

Double hashing is usually better at spreading keys because the jump size depends on the key.

Simple ranking for clustering reduction:

Linear probing: weakest against clustering
Quadratic probing: better
Double hashing: usually strongest

But double hashing needs a good second hash function.

37. Important Formulas in Phase 4

Quadratic Probing

$$H'(x) = (H(x) + i^2) \bmod \text{size}$$

Double Hashing

$$H'(x) = (H1(x) + i * H2(x)) \bmod \text{size}$$

Primary Hash Function Used in Example

$$H1(x) = x \bmod 10$$

Secondary Hash Function Used in Example

$$H2(x) = 7 - (x \bmod 7)$$

Loading Factor

```
lambda = n / size
```

Recommended Rule for Quadratic Probing

```
lambda <= 0.5
```

Part D: Complete C Program

38. Complete C Program for Quadratic Probing and Double Hashing

This program demonstrates both quadratic probing and double hashing.

```
#include <stdio.h>

#define SIZE 10
#define EMPTY 0

int h1(int key) {
    return key % SIZE;
}

int h2(int key) {
    return 7 - (key % 7);
}

void clearTable(int H[]) {
    for (int i = 0; i < SIZE; i++) {
        H[i] = EMPTY;
    }
}

void printTable(int H[]) {
    for (int i = 0; i < SIZE; i++) {
        printf("%d: ", i);
        if (H[i] == EMPTY) {
            printf("EMPTY");
        } else {
            printf("%d", H[i]);
        }
        printf("\n");
    }
}
```

```

    }
}

int insertQuadratic(int H[], int key) {
    int index = h1(key);

    for (int i = 0; i < SIZE; i++) {
        int probeIndex = (index + i * i) % SIZE;

        if (H[probeIndex] == EMPTY) {
            H[probeIndex] = key;
            return probeIndex;
        }
    }

    return -1;
}

int insertDoubleHash(int H[], int key) {
    int index = h1(key);
    int step = h2(key);

    for (int i = 0; i < SIZE; i++) {
        int probeIndex = (index + i * step) % SIZE;

        if (H[probeIndex] == EMPTY) {
            H[probeIndex] = key;
            return probeIndex;
        }
    }

    return -1;
}

int main() {
    int HT[SIZE];

    clearTable(HT);

    int qKeys[] = {23, 43, 13, 27};

    printf("Quadratic probing:\n");

    for (int i = 0; i < 4; i++) {
        int pos = insertQuadratic(HT, qKeys[i]);
        printf("Inserted %d at index %d\n", qKeys[i], pos);
    }
}

```

```

    printTable(HT);

    clearTable(HT);

    int dKeys[] = {5, 25, 15, 35, 95};

    printf("\nDouble hashing:\n");

    for (int i = 0; i < 5; i++) {
        int pos = insertDoubleHash(HT, dKeys[i]);
        printf("Inserted %d at index %d\n", dKeys[i], pos);
    }

    printTable(HT);

    return 0;
}

```

39. Code Explanation: Constants

```

#define SIZE 10
#define EMPTY 0

```

Meaning:

Constant	Meaning
SIZE	The table has 10 slots
EMPTY	A slot with value 0 is considered empty

Important warning:

This program assumes that real keys are not 0.

If 0 can be a real key, then using 0 as EMPTY is dangerous.

A safer real-world program can use a separate status array.

40. Code Explanation: First Hash Function

```
int h1(int key) {  
    return key % SIZE;  
}
```

This function gives the starting index.

Examples:

```
h1(23) = 23 mod 10 = 3  
h1(43) = 43 mod 10 = 3  
h1(95) = 95 mod 10 = 5
```

41. Code Explanation: Second Hash Function

```
int h2(int key) {  
    return 7 - (key % 7);  
}
```

This function gives the jump size for double hashing.

Examples:

```
h2(25) = 7 - (25 mod 7) = 7 - 4 = 3  
h2(15) = 7 - (15 mod 7) = 7 - 1 = 6  
h2(35) = 7 - (35 mod 7) = 7 - 0 = 7
```

Important:

```
h2(key) must not return 0.
```

42. Code Explanation: clearTable

```
void clearTable(int H[]) {  
    for (int i = 0; i < SIZE; i++) {
```

```
        H[i] = EMPTY;
    }
}
```

This function makes every slot empty.

It is used before testing quadratic probing and again before testing double hashing.

43. Code Explanation: printTable

```
void printTable(int H[]) {
    for (int i = 0; i < SIZE; i++) {
        printf("%d: ", i);
        if (H[i] == EMPTY) {
            printf("EMPTY");
        } else {
            printf("%d", H[i]);
        }
        printf("\n");
    }
}
```

This prints every index in the hash table.

If a slot is empty, it prints:

```
EMPTY
```

Otherwise, it prints the stored key.

44. Code Explanation: insertQuadratic

```
int insertQuadratic(int H[], int key) {
    int index = h1(key);

    for (int i = 0; i < SIZE; i++) {
        int probeIndex = (index + i * i) % SIZE;

        if (H[probeIndex] == EMPTY) {
            H[probeIndex] = key;
            return probeIndex;
        }
    }
}
```

```

    }
}

return -1;
}

```

This function inserts a key using quadratic probing.

Important line:

```
int probeIndex = (index + i * i) % SIZE;
```

This is the quadratic probing formula:

$$H'(x) = (H(x) + i^2) \bmod \text{size}$$

If an empty slot is found, the key is stored there.

If no empty slot is found, the function returns `-1`.

45. Code Explanation: insertDoubleHash

```

int insertDoubleHash(int H[], int key) {
    int index = h1(key);
    int step = h2(key);

    for (int i = 0; i < SIZE; i++) {
        int probeIndex = (index + i * step) % SIZE;

        if (H[probeIndex] == EMPTY) {
            H[probeIndex] = key;
            return probeIndex;
        }
    }

    return -1;
}

```

This function inserts a key using double hashing.

Important lines:

```
int index = h1(key);  
int step = h2(key);
```

`index` is the starting position.

`step` is the jump size.

Important formula:

```
int probeIndex = (index + i * step) % SIZE;
```

This matches:

$$H'(x) = (H1(x) + i * H2(x)) \bmod \text{size}$$

46. Program Dry Run Summary

Quadratic probing keys

23, 43, 13, 27

Final quadratic table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	_	_	23	43	_	_	13	27	_

Double hashing keys

5, 25, 15, 35, 95

Final double hashing table:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	_	15	35	_	95	5	_	_	25	_

Part E: Common Mistakes

47. Common Beginner Mistakes in Phase 4

Mistake 1: Starting `i` from 1

Wrong idea:

Start probing with `i = 1`

Correct idea:

Start probing with `i = 0`

Why?

Because `i = 0` checks the original hash index.

Mistake 2: Forgetting the Square in Quadratic Probing

Wrong formula:

$H'(x) = (H(x) + i) \bmod \text{size}$

That is linear probing.

Correct formula:

$H'(x) = (H(x) + i^2) \bmod \text{size}$

Quadratic probing must use `i2`.

Mistake 3: Thinking Quadratic Probing Removes All Problems

Quadratic probing reduces primary clustering, but it does not make the table perfect.

If the table is too full, it can still become slow.

Mistake 4: Letting $H_2(x)$ Return 0

In double hashing, $H_2(x)$ must never be zero.

If $H_2(x) = 0$, then:

$$H'(x) = (H_1(x) + i * 0) \bmod \text{size}$$

The index never changes.

That can cause an infinite loop or insertion failure.

Mistake 5: Forgetting That $H_2(x)$ Depends on the Key

The point of double hashing is that different keys can get different jump sizes.

Wrong idea:

All keys use the same fixed jump.

Correct idea:

The second hash function calculates a jump based on the key.

Mistake 6: Ignoring Table Size

Table size matters.

Poor table sizes can create poor probe sequences.

Double hashing especially depends on good table size and good second hash function design.

Mistake 7: Overfilling the Table

Closed hashing methods need empty slots.

If the table is almost full, probing becomes slow.

Remember:

For quadratic probing, keep $\lambda \leq 0.5$

Mistake 8: Forgetting `mod size`

Wrong idea:

`index + i2`

Correct idea:

`(index + i2) mod size`

The `mod size` part keeps the index inside the valid range.

Part F: Quick Revision

48. Phase 4 Quick Revision Sheet

Concept	Simple meaning
Closed hashing	Store all keys inside the original table
Probe	A position checked after a collision
Linear probing	Move one slot at a time
Primary clustering	Long continuous blocks of filled slots
Quadratic probing	Use square jumps after collision
<code>i²</code>	The square jump used in quadratic probing
Double hashing	Use two hash functions
<code>H1(x)</code>	Starting index
<code>H2(x)</code>	Jump size
<code>H2(x) ≠ 0</code>	Needed so the probe actually moves
Loading factor	How full the table is
<code>lambda = n / size</code>	Loading factor formula

Concept

Simple meaning

`lambda <= 0.5`

Recommended rule for quadratic probing

49. Phase 4 Full Summary

Phase 4 teaches better probing methods for closed hashing.

In Phase 3, you learned linear probing.

Linear probing handles collisions by moving one step at a time.

This is simple, but it creates primary clustering.

Primary clustering means many filled slots form one long block.

That block makes future insertions and searches slower.

Phase 4 improves this using two methods.

The first method is **quadratic probing**.

Quadratic probing uses this formula:

$$H'(x) = (H(x) + i^2) \bmod \text{size}$$

It jumps by square values:

0, 1, 4, 9, 16, ...

This helps reduce primary clustering.

The second method is **double hashing**.

Double hashing uses two hash functions:

$H_1(x) = \text{starting index}$
 $H_2(x) = \text{jump size}$

Its formula is:

$$H'(x) = (H1(x) + i * H2(x)) \bmod \text{size}$$

Double hashing is powerful because different keys can get different jump sizes.

This means even if many keys start at the same index, they may not follow the same probe path.

The most important idea of Phase 4 is:

Do not always move one step after a collision. Use smarter jumps.

50. What You Should Be Able to Explain After Phase 4

After studying this phase, you should be able to explain:

1. Why linear probing creates primary clustering
 2. Why Phase 4 is needed after Phase 3
 3. What quadratic probing means
 4. Why quadratic probing uses square jumps
 5. The formula $H'(x) = (H(x) + i^2) \bmod \text{size}$
 6. Why i starts at 0
 7. How to insert keys using quadratic probing
 8. How to search using quadratic probing
 9. Why quadratic probing reduces primary clustering
 10. What loading factor means
 11. Why quadratic probing should keep $\lambda \leq 0.5$
 12. The average search formulas given for quadratic probing
 13. What double hashing means
 14. Why double hashing uses two hash functions
 15. What $H1(x)$ does
 16. What $H2(x)$ does
 17. Why $H2(x)$ must not be 0
 18. The formula $H'(x) = (H1(x) + i * H2(x)) \bmod \text{size}$
 19. How to insert keys using double hashing
 20. How to search using double hashing
 21. Why double hashing reduces clustering
 22. How linear, quadratic, and double hashing compare
 23. Common mistakes in Phase 4
 24. Why Phase 5 comes next
-

Part G: Mini Practice

51. Mini Practice 1: Quadratic Probing

Use:

```
Table size = 10  
H(x) = x mod 10  
Keys: 12, 22, 32
```

Insert 12

```
12 mod 10 = 2
```

Index is empty.

Store at index .

Insert 22

```
22 mod 10 = 2
```

Index is full.

Quadratic probing:

```
i = 0: 2 → full  
i = 1: 2 + 12 = 3 → empty
```

Store at index .

Insert 32

```
32 mod 10 = 2
```

Index is full.

Quadratic probing:

```
i = 0: 2 → full  
i = 1: 3 → full  
i = 2: 2 + 4 = 6 → empty
```

Store at index .

Final table:

```
Index: 0 1 2 3 4 5 6 7 8 9  
Value: _ _ 12 22 _ _ 32 _ _ _
```

52. Mini Practice 2: Double Hashing

Use:

```
Table size = 10  
H1(x) = x mod 10  
H2(x) = 7 - (x mod 7)  
Keys: 10, 20, 30
```

Insert 10

```
H1(10) = 10 mod 10 = 0
```

Index is empty.

Store at index .

Insert 20

```
H1(20) = 20 mod 10 = 0
```

Index is full.

Calculate:

```
H2(20) = 7 - (20 mod 7)
20 mod 7 = 6
H2(20) = 1
```

Probe:

```
i = 0: 0 → full
i = 1: 0 + 1 * 1 = 1 → empty
```

Store 20 at index 1.

Insert 30

```
H1(30) = 30 mod 10 = 0
```

Index 0 is full.

Calculate:

```
H2(30) = 7 - (30 mod 7)
30 mod 7 = 2
H2(30) = 5
```

Probe:

```
i = 0: 0 → full
i = 1: 0 + 1 * 5 = 5 → empty
```

Store 30 at index 5.

Final table:

```
Index: 0  1  2 3 4 5  6 7 8 9
Value: 10 20 _ _ _ 30 _ _ _ _
```

Main lesson:

20 and 30 both started at index 0, but they moved differently because H2 was different.

53. Final Simple Explanation

Phase 4 is about improving probing.

Linear probing is simple:

If full, check the next slot.

But this can create long blocks of filled positions.

That problem is called:

Primary clustering

Quadratic probing improves this by jumping using squares:

0, 1, 4, 9, 16, ...

Formula:

$$H'(x) = (H(x) + i^2) \bmod \text{size}$$

Double hashing improves it further by using a second hash function for the jump size.

Formula:

$$H'(x) = (H1(x) + i * H2(x)) \bmod \text{size}$$

So the heart of Phase 4 is:

After a collision, do not always move one step.
Use smarter jumps to spread keys better.

Phase 5 comes next because after learning collision methods, you must learn how to design good hash functions and choose the best method.